Lecture 15

Resource Management

Example Code and Exercises

`https://github.com/eecs498-software-design/lecture`

Recall: Exception Safety

- This version <u>almost</u> provides the **strong guarantee**, except it <u>leaks memory</u>.
- What's missing?

```
template <typename T>
class vector {
private:
  T *elts;          // points to dynamic array
  size_t capacity;  // capacity of elts array
  size_t elts_size; // number of slots used in elts

public:
  vector & operator=(const vector &rhs) {
    T *temp = new T[rhs.capacity]; // may throw (bad_alloc)

    for (int i = 0; i < rhs.elts_size; ++i) {
      temp[i] = rhs.elts[i]; // may throw (assn op for T)
    }

    delete[] elts;
    elts = temp;
    capacity = rhs.capacity;
    elts_size = rhs.elts_size;

    return *this;
  }
}
```

C++

Resource Management

Option 1a

Use `try/catch`.

- Idea: Temporarily catch the exception so we can free the temporary memory, then rethrow it.

```C++
template <typename T>
class vector {
private:
  T *elts;          // points to dynamic array
  size_t capacity;  // capacity of elts array
  size_t elts_size; // number of slots used in elts

public:
  vector & operator=(const vector &rhs) {
    T *temp = nullptr;
    try {
      temp = new T[rhs.capacity]; // may throw (bad_alloc)
      for (int i = 0; i < rhs.elts_size; ++i) {
        temp[i] = rhs.elts[i]; // may throw (assn op for T)
      }
      delete[] elts;
      elts = temp;
      capacity = rhs.capacity;
      elts_size = rhs.elts_size;
    }
    catch (...) {
      delete[] temp;
      throw; // rethrow the exception
    }
    return *this;
  }
};
```

C++

Resource Management

Option 1b

Use `try/finally`.

- Idea: A `finally` block is **always** run when exiting the matching `try` block.
- However – C++ doesn't actually have a finally keyword.

```
template <typename T>
class vector {
private:
  T *elts;          // points to dynamic array
  size_t capacity;  // capacity of elts array
  size_t elts_size; // number of slots used in elts

public:
  vector & operator=(const vector &rhs) {
    T *temp = nullptr;
    try {
      temp = new T[rhs.capacity]; // may throw (bad_alloc)
      for (int i = 0; i < rhs.elts_size; ++i) {
        temp[i] = rhs.elts[i]; // may throw (assn op for T)
      }
      delete[] elts;
      elts = temp;
      capacity = rhs.capacity;
      elts_size = rhs.elts_size;
    }
    finally {
      delete[] temp;
    }
    return *this;
  }
};
```

C++

Resource Management

Option 1b

Use `try/finally`.

- Idea: A `finally` block is **always** run when exiting the matching `try` block.
- However – C++ doesn't actually have a finally keyword.

```
template <typename T>
class vector {
private:
  T *elts;          // points to dynamic array
  size_t capacity;  // capacity of elts array
  size_t elts_size; // number of slots used in elts

public:
  vector & operator=(const vector &rhs) {
    T *temp = nullptr;
    try {
      temp = new T[rhs.capacity]; // may throw (bad_alloc)
      for (int i = 0; i < rhs.elts_size; ++i) {
        temp[i] = rhs.elts[i]; // may throw (assn op for T)
      }
      delete[] elts;
      elts = temp;
      capacity = rhs.capacity;
      elts_size = rhs.elts_size;
    }
    finally {
      delete[] temp;
    }
    return *this;
  }
};
```

C++

Resource Management

Option 2a

Use a smart pointer.

- Idea: The temp array is now declared as a `std::unique_ptr`. Its destructor automatically deletes the array when it goes out of scope.
- Release the smart pointer once we've safely finished any throwing steps.

```C++
template <typename T>
class vector {
private:
  T *elts;          // points to dynamic array
  size_t capacity;  // capacity of elts array
  size_t elts_size; // number of slots used in elts

public:
  vector & operator=(const vector &rhs) {
    // may throw (bad_alloc)
    std::unique_ptr<T[]> temp(new T[rhs.capacity]);
    for (int i = 0; i < rhs.elts_size; ++i) {
      temp[i] = rhs.elts[i]; // may throw (assn op for T)
    }

    delete[] elts;
    elts = temp.release(); // transfer ownership to elts

    capacity = rhs.capacity;
    elts_size = rhs.elts_size;
    return *this;
  }
};
```

C++

Resource Management

Option 2b

Use two smart pointers.

- Idea: Both elts and temp are `std::unique_ptrs`.
- Each manages an array "behind the scenes". `temp` will always be cleaned up.
- But, if (and only if) the copy is successful, we quickly swap them.

```
template <typename T>
class vector {
private:
  std::unique_ptr<T[]> elts; // manages a dynamic array
  size_t capacity;  // capacity of elts array
  size_t elts_size; // number of slots used in elts

public:
  vector & operator=(const vector &rhs) {
    // may throw (bad_alloc)
    std::unique_ptr<T[]> temp(new T[rhs.capacity]);
    for (int i = 0; i < rhs.elts_size; ++i) {
      temp[i] = rhs.elts[i]; // may throw (assn op for T)
    }

    // The new (2x) array now lives as elts.
    // The old array becomes temp and will go out of scope.
    std::swap(elts, temp); // Important! swap is no-throw

    capacity = rhs.capacity;
    elts_size = rhs.elts_size;
    return *this;
  }
};
```

C++

Resource Management

Option 2c

The Copy-Swap Idiom.

- Idea: Use the copy ctor to make a `temp` copy of the whole vector.
- If (and only if) the copy is successful, we trade our pointers with `temp`.
- The destructor on `temp` always cleans up, either a failed copy or the original, old data.

```C++
template <typename T>
class vector {
private:
  T *elts; // points to dynamic array
  size_t capacity;  // capacity of elts array
  size_t elts_size; // number of slots used in elts

public:
  // Trade the guts of the vectors by trading pointers.
  void swap(vector &other) {
    using std::swap; // enable ADL (a random C++ thing)
    swap(elts, other.elts); // unqualified call for ADL
    swap(capacity, other.capacity);
    swap(elts_size, other.elts_size);
  }

  vector & operator=(vector rhs) { // copy - pass by value
    swap(rhs); // swap – Important! Never throws.
    return *this;
    // cleanup - rhs dtor. either deletes any temp data
    // if the copy ctor throws, or old data if successful.
  }
};
```

C++

Resource Management

- A resource is anything that must be:
  - Acquired
  - Released

- Presumably:
  - There are a finite number of resources
  - Forgetting to acquire/free them causes problems
  - There isn't a simple fix that the compiler or runtime environment can completely handle for you.

Resource Management

- Using a **manual try/catch** ensures cleanup on both paths, but the complexity can explode with additional resources or error paths.

```
function readConfigWithCatch(path: string): string {
  const fd = fs.openSync(path, "r");
  const buffer = Buffer.alloc(1024);
  try {
    fs.readSync(fd, buffer);
    fs.closeSync(fd); // clean up on happy path
    return buffer.toString();
  } catch (e) {
    fs.closeSync(fd); // also clean up on error path
    throw e;
  }
}
```

Resource Management

- Many languages support a **finally** block at the end of a `try/catch`.
- It is guaranteed to run no matter what when you leave the scope.

```
function readConfigWithFinally(path: string): string {
  const fd = fs.openSync(path, "r");
  const buffer = Buffer.alloc(1024);
  try {
    fs.readSync(fd, buffer);
    return buffer.toString();
  } finally {
    fs.closeSync(fd); // clean up on every path
  }
}
```

Structured Resource Management

Many languages support automatic cleanup based on lexical scope or object lifetimes.

Examples:

- C++: Destructors (RAII)
- Java: “Try-with-resources” blocks
- Python: `with` statements
- Typescript: `using` declarations

```
class FileHandle implements Disposable {
  private fd: number;
  private closed = false;

  constructor(path: string, flags: string) {
    this.fd = fs.openSync(path, flags);
  }

  [Symbol.dispose](): void {
    if (!this.closed) {
      fs.closeSync(this.fd);
      this.closed = true;
    }
  }
}

function readConfig(path: string): string {
  using file = new FileHandle(path, "r");
  return file.readAll();
  // file[Symbol.dispose]() called
  // automatically here!
}
```

In Typescript, declaring something with `using` (rather than `let`/`const`) ensures that a special `[Symbol.dispose()]` function will be called when it goes out of scope.

Garbage Collection

- The GC periodically frees memory for any objects no longer reachable from starting points like the current stack, task queue, global variables, etc.

```
function gcExample() {
  let obj = { data: "hello" };  // Object is reachable via `obj`
  console.log(obj.data);

  obj = null;  // Now nothing references the object

  // GC can reclaim { data: "hello" } (eventually, when it runs)
}
```

Garbage Collection

- Closures can also keep an object alive via implicit references.

```
function makeFormatter(postfix: string) {
  return (s: string) => s + postfix;
  // The anonymous function returned keeps the postfix alive
}

function processData(data: { items: string[] }) {
  const formatter = makeFormatter(" cats"); // " cats" kept alive past this line
  data.items = data.items.map(item => formatter(item));
  // when formatter goes out of scope, now " cats" is no
  // longer reachable and may be garbage collected
}

const result = processData({ items: ["1", "2", "3", "4", "5"] });
```

Closures - Review

- A closure "captures" references from the stack frame in which it is defined.
- It does not create its own references to the current object.

```
interface PersonData {
  name: string; age: number; email: string;
}
class Person {
  constructor(private data: PersonData) { }
  public setData(newData: PersonData): void {
    this.data = newData;
  }
  public getEmailUpdater() {
    return (newEmail: string) => {
      this.data.email = newEmail;
    };
  }

  public getNameUpdater() {
    const data = this.data;
    return (newName: string) => {
      data.name = newName;
    };
  }
}
```

Captures `this`. The updater will work.

Captures local `data`. Will always refer to the `PersonData` object that was present at the time this was called, which may be out of date.

```
function main() {
  const person = new Person({
    name: "Eddie", age: 6,
    email: "eddiecat@cat.com"
  });

  const updateEmail = person.getEmailUpdater();
  const updateName = person.getNameUpdater();

  person.setData({
    name: "Eddie", age: 7,
    email: "eddiecat@cat.com"
  });

  updateEmail("eddieTheCat@cat.com");
  updateName("Edward");

  // What are the person's name and email now?
}
```

`"Eddie", 7, "eddieTheCat@cat.com"`

Garbage Collection and Memory Leaks

- Does a garbage collector prevent memory leaks?
  - Yes - if the memory leak was due to memory that is unreachable.
  - No – if the memory is still reachable, but is not really being used.

- Example:

```
class Logger {
  private logs: string[] = [];

  log(message: string): void {
    this.logs.push(`[${new Date().toISOString()}] ${message}`);
  }
}
```

This can easily cause a memory leak! In a long-running program, we can accumulate a huge array of logs. Perhaps we should clear them periodically?

Memory Leaks, References, and Design

```ts
interface Dataset {
  name: string;
  values: number[];
  metadata: object;
}

class Report {
  constructor(private source: Dataset) { }

  getSum(): number {
    return this.source.values.reduce(
      (a, b) => a + b, 0);
  }

  display(): void {
    console.log(`Sum of ${this.source.name}:
${this.getSum()}`);
  }
}
```

It may be unintuitive that a "report" takes up enough memory to store the entire original dataset.

Could this be precomputed and part of the metadata instead?

```ts
function generateReports() {
  const reports: Report[] = [];

  for (let i = 0; i < 100; i++) {
    // Create a large dataset for each report
    const dataset: Dataset = {
      name: `Dataset ${i}`,
      values: new Array(10000).fill(i),
      metadata: {
        created: new Date(),
        tags: ["foo"]
      }
    };

    reports.push(new Report(dataset));
  }

  return reports;
}
```

Lifetimes, References, and Design

```
const people: Person[] = [
  { name: "Alice", history: new Array(10000).fill("action") },
  { name: "Bob", history: new Array(10000).fill("action") },
];
```

```
function createGreeterFunction(person: Person) {
  return () => {
    console.log(`Hello, ${person.name}!`);
  };
}
```

It may be unintuitive that the anonymous function created here keeps alive the full history array via a closure.

Lifetimes, References, and Design

```
const people: Person[] = [
  { name: "Alice", history: new Array(10000).fill("action") },
  { name: "Bob", history: new Array(10000).fill("action") },
];
```

```
function createGreeter(person: Person) {
  const name = person.name;
  return () => {
    console.log(`Hello, ${name}!`);
  };
}
```

This version attempts to ensure that we don't capture the full person data, just the local reference to their name.

This may work, depending on the runtime environment optimizations - but sometimes a closure includes the whole stack frame and keeps alive all references, even those we don't use in the closure (the `person` parameter here).

Lifetimes, References, and Design

```
const people: Person[] = [
  { name: "Alice", history: new Array(10000).fill("action") },
  { name: "Bob", history: new Array(10000).fill("action") },
];
```

```
function createGreeter(name: string) {
  return () => {
    console.log(`Hello, ${name}!`);
  };
}
```

```
function createGreeter(person: Person) {
  const name = person.name;
  return buildGreeter(name);
}
```

This version uses an “extra stack frame” to ensures that nothing can be captured except the name reference.

```
function buildGreeter(name: string) {
  return () => {
    console.log(`Hello, ${name}!`);
  };
}
```

Resource Management

- Is there a resource leak here?

```
interface GameEventObserver {
  onEvent(): void;
}

class GameWithEvents {
  private observers = new Set<Observer>();

  addObserver(obs: GameEventObserver) {
    this.observers.add(obs);
    return () => { // return an unsubscribe fn
      this.observers.delete(obs);
    };
  }

  removeObserver(callback: GameEventObserver) {
    this.observers.delete(callback);
  }
}
```

The observer subscription is a resource. It must be acquired and we must also release it.

```
class ThemedGameUI {
  private game: Game;
  private theme: Theme;

  constructor(game: Game, opts: ThemeOptions) {
    this.game = game;
    this.theme = new Theme(opts);
    game.addObserver(this);
  }

  private onEvent(): void { /* ... */ }
}

class Driver {
  private game : GameWithEvents;
  private ui : GameUI;

  public change_theme() {
    // called when the user clicks a button
    // Assume they enter some options for a theme
    const opts = get_opts();
    this.ui = new ThemedGameUI(game);
    ui.render();
  }
```

It is not enough that we switch to a new `ThemedGameUI` instance, assuming the old one will be GC'ed. It won't! The observer keeps it alive, and in fact it even keeps receiving events and updating itself!

Resource Management

- Is there a resource leak here?

```
interface GameEventObser
  onEvent(): void;
}

class GameWithEvents {
  private observers = new Set<Observer>();

  addObserver(obs: GameEventObserver) {
    this.observers.add(obs);
    return () => { // return an unsubscribe fn
      this.observers.delete(obs);
    };
  }

  removeObserver(callback: GameEventObserver) {
    this.observers.delete(callback);
  }
}
```

In other contexts, we could use this with a local `using` declaration that automatically calls `Symbol.dispose()` when it goes out of scope.

```
class ThemedGameUI implements Disposable {
  private game: Game;
  private theme: Theme;
  constructor(game: Game, opts: ThemeOptions) {
    this.game = game;
    this.theme = new Theme(opts);
    game.addObserver(this);
  }
  private onEvent(): void { /* ... */ }
  public [Symbol.dispose](): void {
    game.removeObserver(this);
  }
}

class Driver {
  private game : GameWithEvents;
  private ui : GameUI;

  public change_theme() {
    // called when the user clicks a button
    // Assume they enter some c
    const opts = get_opts();
    this.ui[Symbol.dispose]();
    this.ui = new ThemedGameUI(game);
    ui.render();
  }
}
```

Create a special function that cleans up resources held by the UI.

Make sure to call it when we're done with the UI!